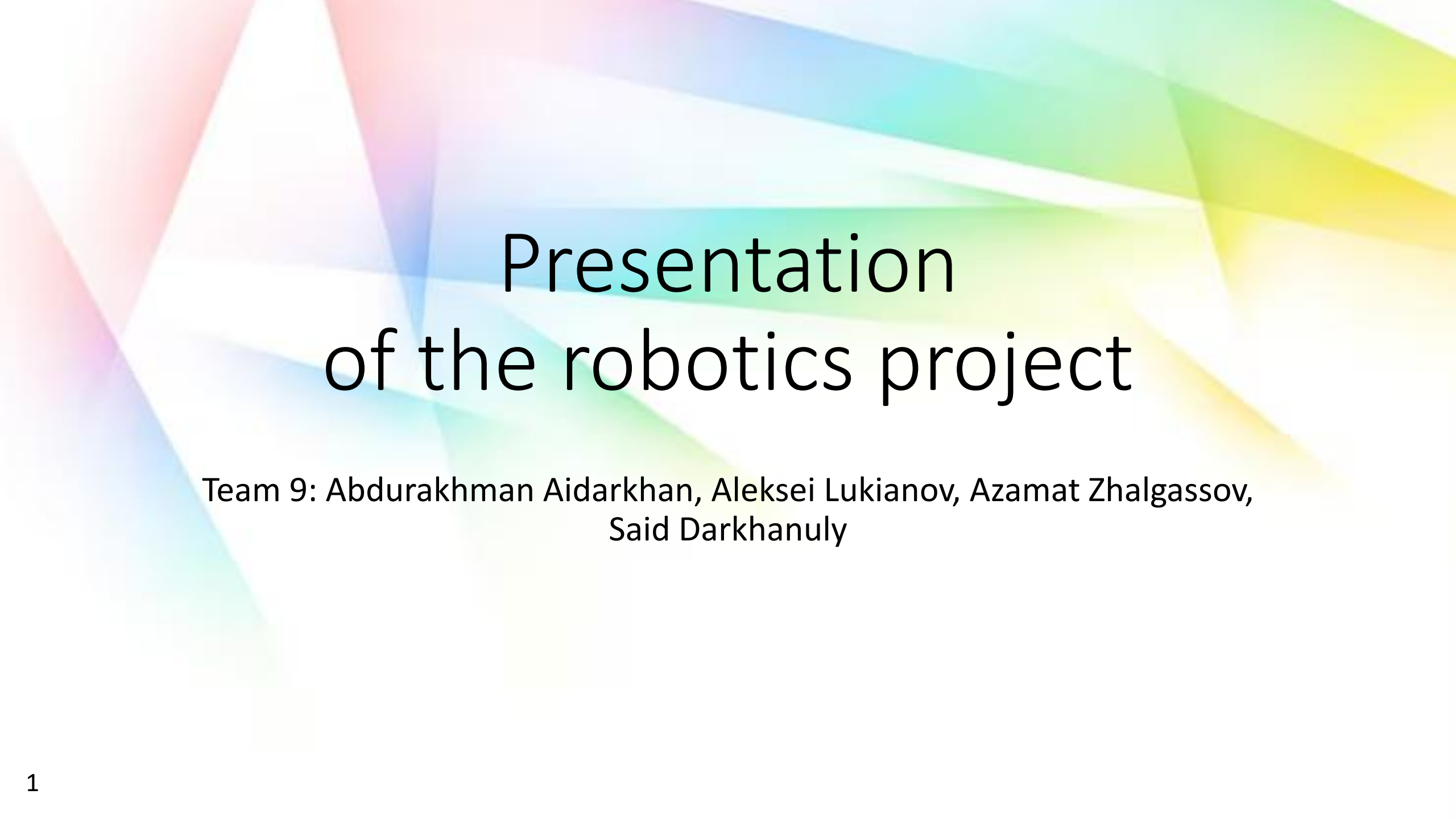




Presentation of the robotics project

Team 9: Abdurakhman Aidarkhan, Aleksei Lukianov, Azamat Zhalgassov,
Said Darkhanuly

Introduction

Our topic is development of autonomous rover which is able to detect obstacles using LIDAR and avoid them. Entire decision-making logic is written on Python. The rover is also capable of solving maze using the "right-hand rule".

Our project goal: the rover successfully avoid obstacles and solve maze autonomously

Essential theory regarding used hardware

List of used hardware components:

1. Waveshare UGV02
2. Nvidia Jetson Xavier NX
3. RPLidar A1
4. AGM Battery 12V 7.2Ah
5. 16AWG 32V 20A Fuse Holder with Wire and 3A fuse
6. Wago Lever Connection Clamp
7. 1 meter DC Extension Cable 12V 18AWG 5.5 mm x 2.5 mm
8. F2 faston terminals
9. 3 batteries 3.7V 3200mAh Flat Top NIMH
10. USB A to Micro USB 2.0 Cable

Waveshare UGV02

UGV02 is a 4WD six-wheel mobile robot chassis with good off-road performance and open-source code that supports customization development. It supports external host computers (Raspberry Pi, Jetson Nano, RDK X3, etc.), and the host computer directly communicates with the ESP32 slave computer.

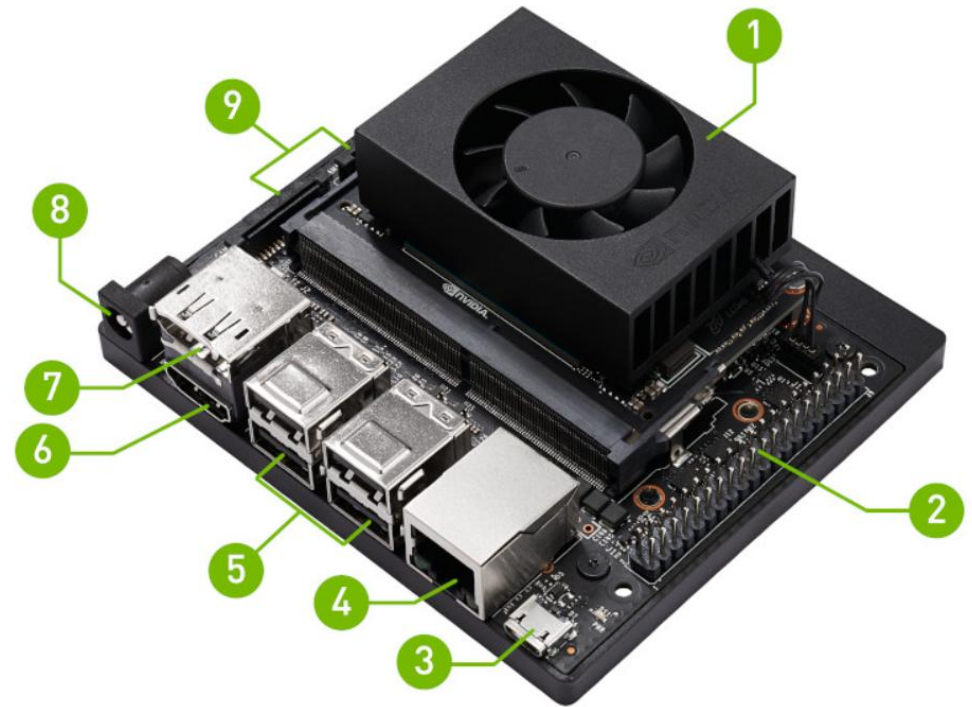
This mobile robot basis of our project, helping us to implement the obstacle avoidance / maze solving logic.



Nvidia Jetson Xavier NX

NVIDIA Jetson Xavier NX is a single-board embedded computing module with a CPU, GPU and accelerators designed for high-performance computing.

This host-computer reads LIDAR data, executing Python code with decision-making logic and sends http-requests to the ESP32 module (Waveshare board) over Wi-Fi



RPLidar A1



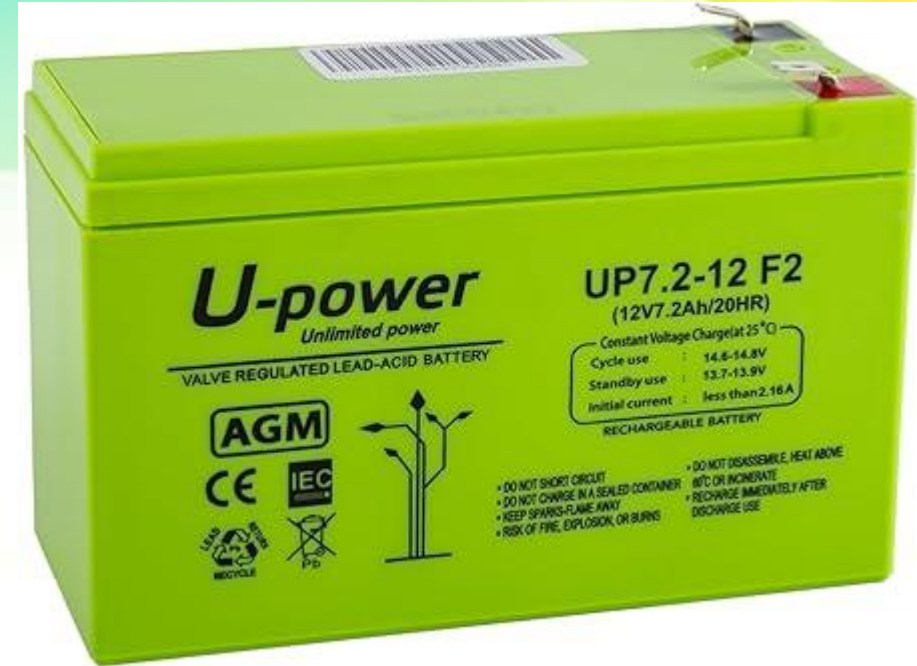
RPLIDAR A1 is a 360 degree 2D laser scanner (LIDAR) solution. The system can perform 360 degree scan within 6 meter range. The produced 2D point cloud data can be used in environment modeling.

We use this device to scan environment and detect obstacles, which are closer than determined safe distance in order to avoid them.

AGM Battery 12V 7.2Ah

An AGM battery is a type of rechargeable battery. It uses lead-acid technology, and the electrolyte is held in a special fiberglass material, making it spill-resistant and maintenance-free.

This battery serves as external source of power for Nvidia Jetson.



16AWG 32V 20A Fuse Holder with Wire and 3A fuse

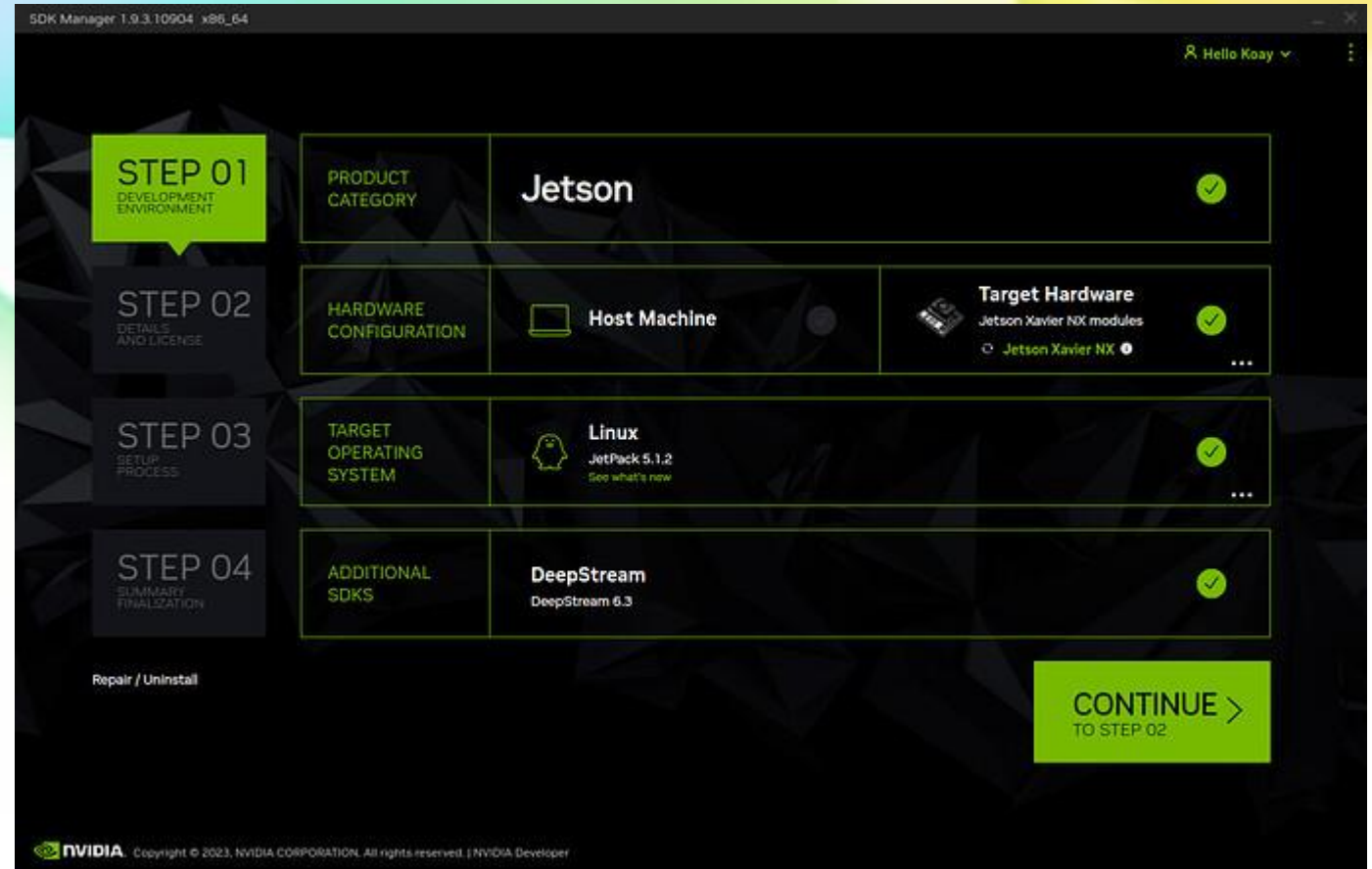
Fuse serves as short circuit protection to prevent damage to the host-computer.

In our case we used 3A fuse



Nvidia Jetson Xavier NX flash

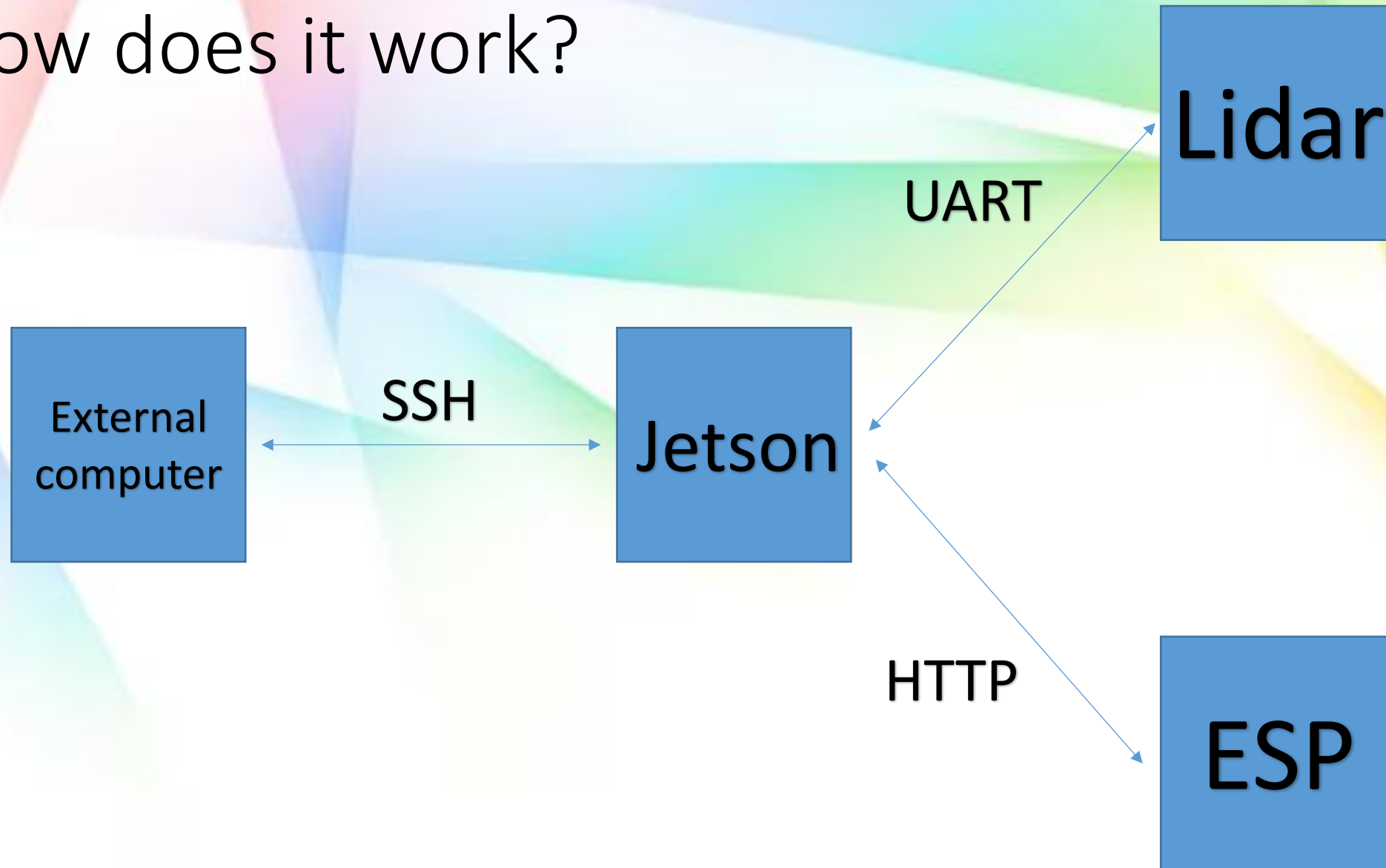
First, we installed Ubuntu system in order to flash Nvidia Jetson Xavier NX. In this process, we switched Nvidia Jetson to recovery mode and then used USB A to Micro USB 2.0 Cable.



Used software

1. Python environment
2. Rplidar library (used for iterative scanning of the area)
3. Requests library (used for sending post http-requests to 80 port of ESP32 module via Wi-Fi)
4. Time library (used for lidar stops between moving commands in order not to overflow http and lidar buffer)
5. SSH protocol (for executing python program from external computer)

How does it work?



HTTP requests

For moving we use command which is represented as a json, containing number of the command and additional parameters. In this case speed values of left and right motors

Http requests are sent to ESP module via url with esp ip address

```
url = "http://10.222.197.188/js"
data = {"T": 1, "L": 0.0, "R": 0.0}

PORT_NAME = '/dev/ttyUSB0'
lidar = RPLidar(PORT_NAME)

SAFE_DIST = 280
```

```
def turn_right():
    data["L"] = 0.08
    data["R"] = -0.08
    r = requests.post(url, json = data, timeout = 10)
    print("Status:", r.status_code)
```

Lidar scans

Lidar does 1 360 spin,
then dots are divided
by sectors and lidar
stops while program
choose direction of
movement in order not
to overfill lidar's buffer

```
try:
    while True:
        scan = next(lidar.iter_scans())
        front = []
        right = []

        for (_, angle, distance) in scan:

            angle = norm_angle(angle)

            if -15 <= angle <= 15:
                front.append(distance)
            elif 65 <= angle <= 115:
                right.append(distance)
```

```
lidar.stop()
lidar.stop_motor()

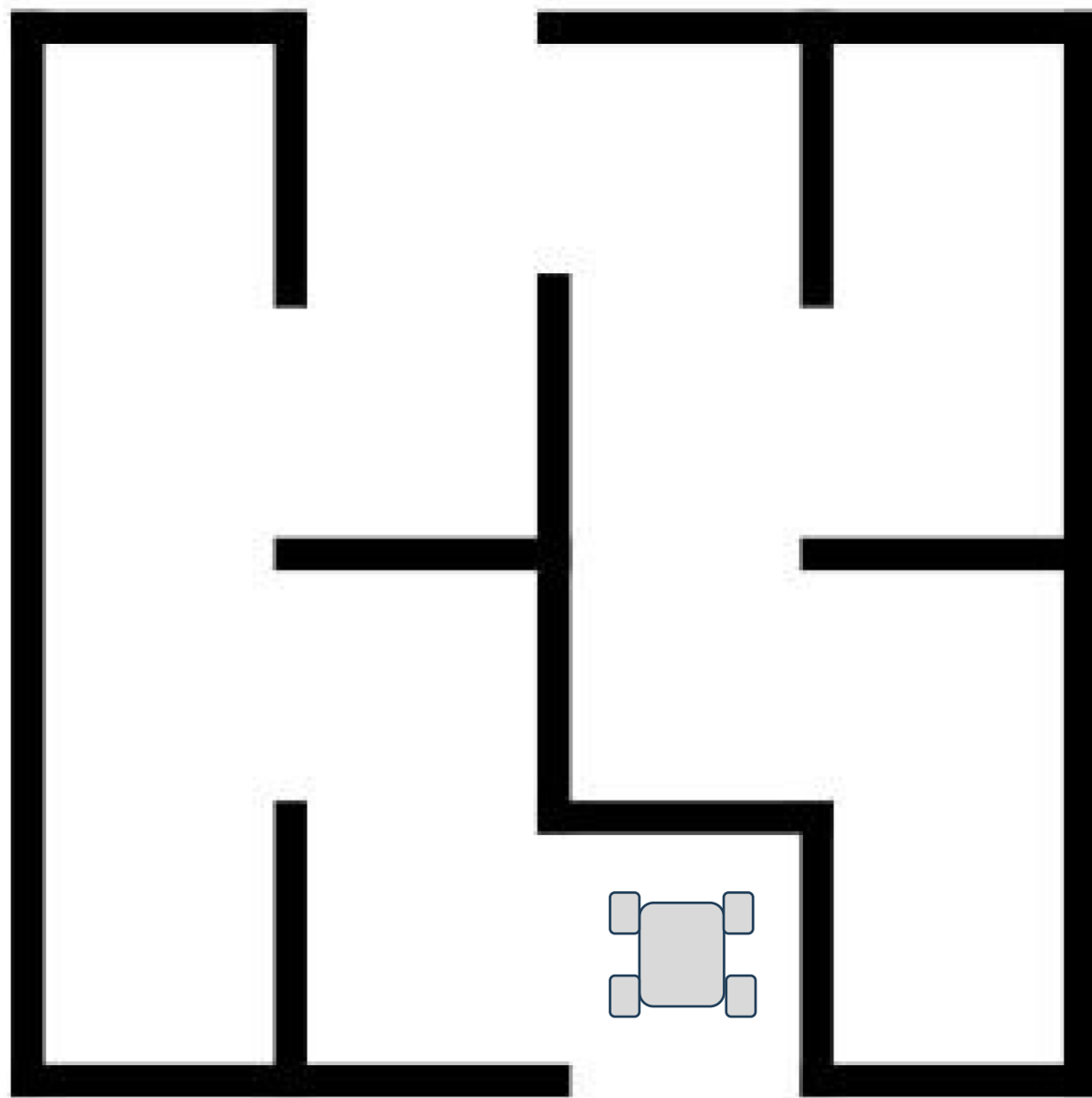
if right_min > SAFE_DIST:
    turn_right()
    time.sleep(3)
    move_forward_after_right()
    pass
elif front_min > SAFE_DIST:
    move_forward_after_left()
    pass
else:
    turn_left()
    pass

time.sleep(3)
lidar.start_motor()
```

Scenarios of the rover

1. Maze-solving robot:

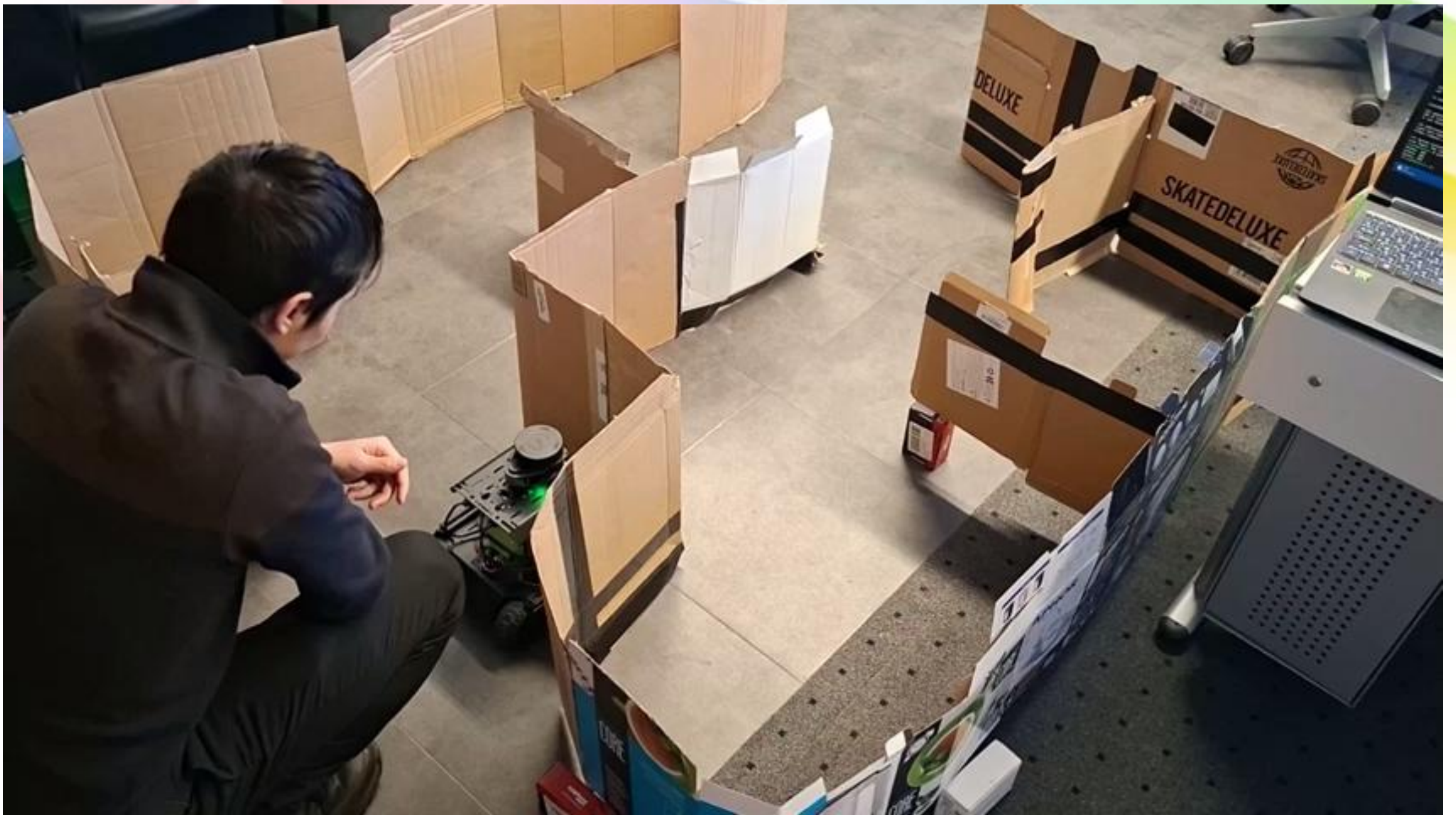
Robot is placed at the start point of the maze and has to find the exit. For this scenario is used right-handed algorithm. If there is a free space on the right, robot goes right for 1 cell, else if there is an obstacle on the right, it moves forward for 1 cell, otherwise turns left and doesn't move.



DISCLAIMER

Due to not ideal surfaces and maze dimensions, robot can't turn exactly for 90 degrees, so human support is needed





Scenarios of the rover

2. Obstacle avoider:

Robot has a goal to move forward. If it sees an obstacle, the emergency stop command is sent and the rover decides where is more free space (on the left or on the right) and moves there, then turns and continues moving in original direction

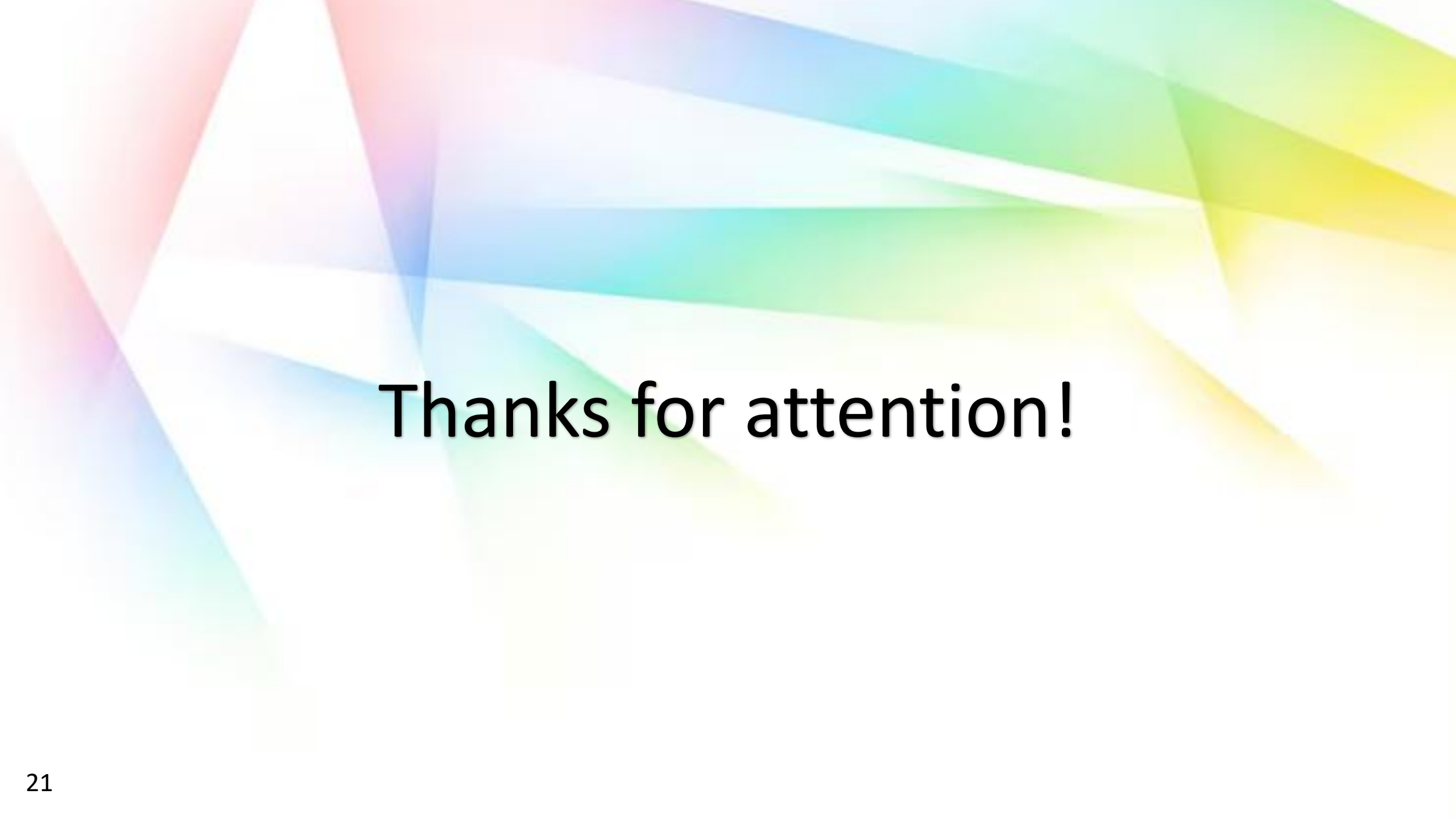


Outlook and conclusion - Challenges, what can be improved

Our initial goal was successfully reached and the mobile robot is capable of maze solving and obstacle avoiding. However, we faced some challenges, such as not ideal surface.

In addition, as we mentioned in our pager, path memorization and optimization could be added for more efficient maze passing without environment scanning.





Thanks for attention!